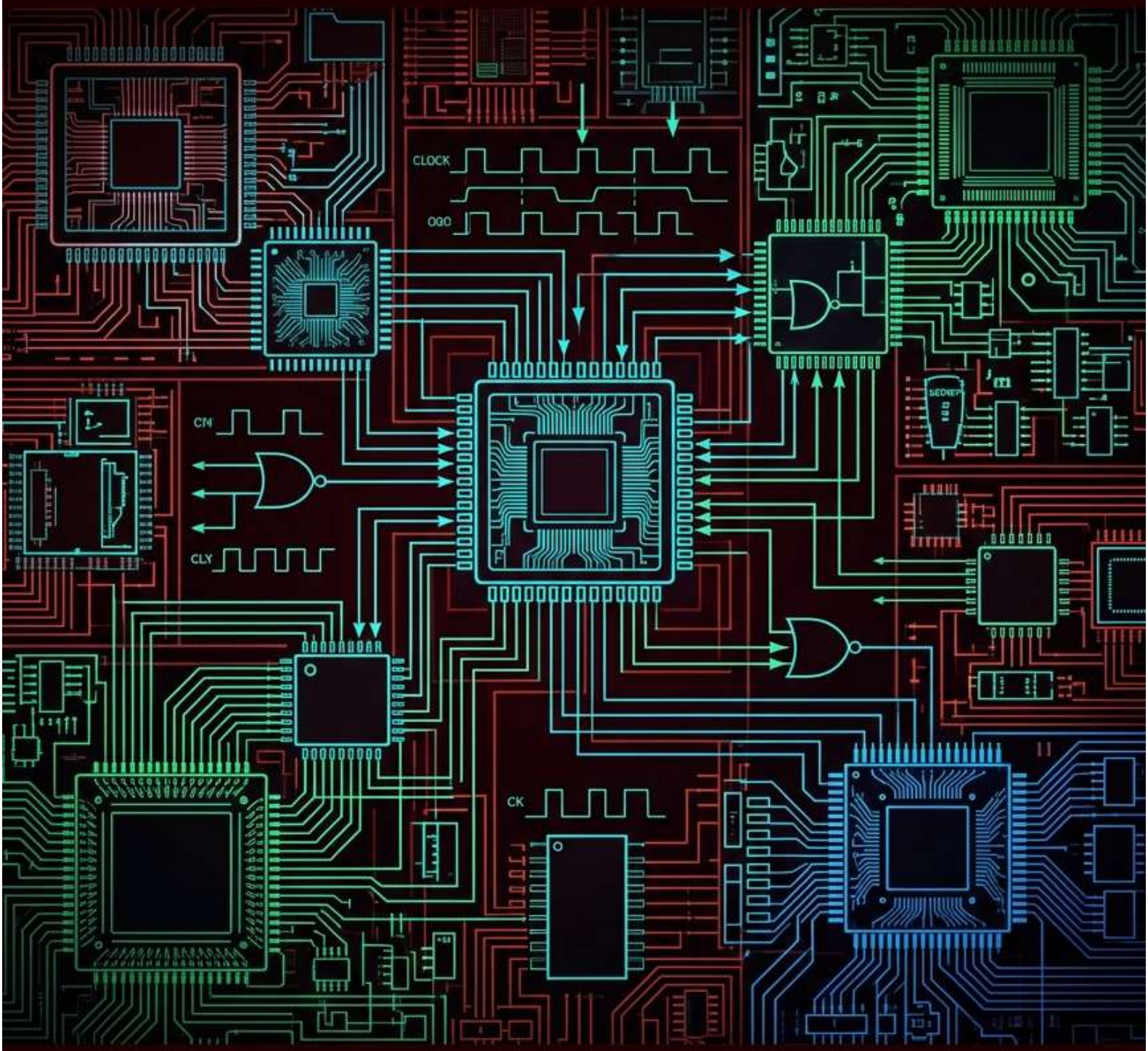
SDC Constraints – Complete Practical & Theoretical Guide

From Basics to Industry-Level Implementation



Banothu Bapuji, M.Tech VLSI System Design

SDC CONSTRAINTS

Complete Practical & Theoretical Guide

From Basics to Industry-Level Implementation

Author: Banothu Bapuji

M.Tech VLSI System Design

Specialization: ASIC Design | Static Timing Analysis | Physical Design

Synopsys Design Constraints (SDC) | STA | ASIC/FPGA Design Flows

Table of Contents

Section 1 Introduction to Timing Constraints

1.1 What is SDC?
1.2 Why Constraints are Required
1.3 Role in Synthesis and STA
1.4 Real-World Analogy

Section 2 Units and Environment Setup

2.1 set_units
2.2 Operating Conditions (PVT)
2.3 Wireload Models vs RC Extraction

Section 3 Clock Constraints

3.1 create_clock
3.2 create_generated_clock
3.3 Clock Latency
3.4 Clock Uncertainty
3.5 Clock Transition
3.6 Ideal vs Propagated Clocks
3.7 Clock Gating Considerations

Section 4 Input Constraints

4.1 set_input_delay
4.2 set_input_transition
4.3 set_driving_cell

Section 5 Output Constraints

5.1 set_output_delay
5.2 set_load

Section 6 Timing Exceptions

6.1 set_false_path
6.2 set_multicycle_path
6.3 set_max_delay / set_min_delay
6.4 When to Use and When NOT to Use

Section 7 Clock Domain Crossing (CDC)

7.1 set_clock_groups
7.2 Asynchronous Clocks

Section 8 Design Rule Constraints (DRC)

8.1 set_max_transition
8.2 set_max_capacitance
8.3 set_max_fanout

Section 9 Advanced Constraints

9.1 set_case_analysis
9.2 set_disable_timing
9.3 Path Grouping
9.4 Hierarchical Constraints

Section 10 Variations and Derates (OCV/AOCV/POCV)

Section 11 Pre-Layout vs Post-Layout Constraints

Section 12 Constraint Validation and Debugging
Section 13 Common Mistakes and Best Practices
Section 14 Complete Sample SDC File
Section 15 Interview Questions & Answers

Section 1: Introduction to Timing Constraints

1.1 What is SDC?

SDC stands for Synopsys Design Constraints. It is an industry-standard file format (with a .sdc extension) used to specify timing, area, and power constraints for digital ASIC and FPGA designs. SDC is based on the Tcl (Tool Command Language) scripting language and is supported by virtually every industry EDA tool — including Synopsys DC, Cadence Genus, Xilinx Vivado, Intel Quartus, and many others.

An SDC file communicates the designer's intent to the synthesis and place-and-route tools. It tells the tool: how fast the clock runs, what the input/output timing requirements are, which paths are timing-critical, and which paths can be safely relaxed or ignored.

 NOTE SDC is not a proprietary format of Synopsys alone — it has become the universal constraint language for all major EDA flows. Any constraint written in SDC is valid Tcl syntax.

1.1.1 Key Characteristics of SDC

- Text-based, human-readable file
- Uses standard Tcl syntax and commands
- Works across synthesis, STA, and layout tools
- Defines clocks, I/O timing, exceptions, and environment
- Extensible — can contain Tcl loops, variables, and conditionals

1.2 Why Constraints are Required

Modern digital designs consist of millions of gates and flip-flops. Without constraints, a synthesis or STA tool has no knowledge of:

- How fast the design must operate (clock frequency)
- What the timing environment outside the chip looks like
- Which signals change on which edges
- Which timing paths are real and which are irrelevant

Without constraints, the tool will either optimize for minimum area (ignoring speed), or report millions of irrelevant violations making debug impossible. Constraints give the tool the context it needs to optimize correctly and report accurately.

 WARNING Under-constrained designs lead to silicon failures. Over-constrained designs lead to area/power waste and impossibly long implementation runs. Getting constraints right is a critical engineering skill.

1.2.1 Consequences of Missing Constraints

Missing Constraint	Consequence
No clock definition	Tool treats design as combinational; no setup/hold checks
No input delay	Paths from inputs optimized without real context
No output delay	Paths to outputs may be too slow for the external interface
No false paths	Fake violations reported on asynchronous resets, test mux paths

No clock uncertainty	Hold violations at silicon due to optimistic assumption

1.3 Role in Synthesis and STA

1.3.1 Role in Synthesis

During synthesis, the tool maps RTL (Verilog/VHDL) to gate-level netlists using cells from a standard cell library. The SDC constraints drive the synthesis engine to:

1. Select faster (but larger) cells on critical paths
2. Choose multi-stage buffers to meet transition constraints
3. Perform retiming or restructuring to meet timing
4. Calculate timing slack and report violations

1.3.2 Role in STA (Static Timing Analysis)

In STA, the tool performs exhaustive timing checks on all paths in the design without running actual simulation. SDC constraints define:

- Which paths to check (clocked paths, I/O paths)
- What the required arrival time (RAT) is for each path
- Which paths should be excluded (false paths, multicycle)
- Clock relationships for cross-domain checks

> ✓**TIP** STA tools like PrimeTime use the SDC to build a timing graph. The accuracy of timing sign-off depends entirely on the correctness of the SDC file.

1.3.3 SDC in the ASIC Design Flow

```
RTL Design (Verilog/VHDL)
     |
     v
Synthesis (DC/Genus) <--- SDC (clocks, I/O, exceptions)
     |
     v
Gate-Level Netlist
     |
     v
Place & Route (ICC2/Innovus) <--- SDC + SPEF (parasitics)
     |
     v
STA Sign-off (PrimeTime) <--- SDC + SPEF (post-route)
     |
     v
GDSII (Tape-out)
```

1.4 Real-World Analogy

Imagine you are a highway engineer building a road. Without specifications, you don't know:

- How fast cars should travel (clock frequency = speed limit)
- Where cars are entering and exiting (input/output delays = on-ramps/off-ramps)
- Which roads are closed for maintenance (false paths = closed roads)
- Whether roads have potholes (clock uncertainty = road quality variation)

SDC constraints are the specification sheet for your digital highway. They define the rules that the EDA tool (road builder) must follow to ensure correct and timely operation.

SECTION SUMMARY

- ✓ SDC = Synopsys Design Constraints; industry-standard Tcl-based format for timing constraints
- ✓ Without constraints, synthesis and STA tools cannot operate correctly
- ✓ SDC drives synthesis optimization and defines the timing environment for STA
- ✓ SDC is used throughout the ASIC/FPGA design flow from synthesis to sign-off

Section 2: Units and Environment Setup

Before writing any timing constraints, it is essential to establish the units and operating environment. These settings affect how all subsequent constraints are interpreted.

2.1 set_units

2.1.1 Theory

The set_units command defines the units for time, capacitance, resistance, voltage, current, and power used throughout the SDC file. If not specified, the tool inherits units from the technology library (.lib file). It is best practice to explicitly define units to avoid ambiguity.

2.1.2 Syntax

```
set_units
    [-time <unit>]
    [-capacitance <unit>]
    [-resistance <unit>]
    [-voltage <unit>]
    [-current <unit>]
    [-power <unit>]
```

2.1.3 Common Units

Parameter	Common Value	Description
-time	1ns or 1ps	Time unit for all delays
-capacitance	1pF or 1fF	Capacitance for load/transition
-resistance	1kOhm or 1Ohm	Resistance for RC models
-voltage	1V	Voltage for operating conditions
-current	1mA or 1uA	Current unit
-power	1mW or 1uW	Power unit

2.1.4 Real Example

```
# Set units explicitly at the top of every SDC file
set_units -time ns -capacitance pF -resistance kOhm -voltage V -current mA

# After this command, all delay values are in nanoseconds
# e.g., create_clock -period 10 means 10 ns
```

 WARNING Always place set_units at the very top of the SDC file. If your library uses ps and your SDC uses ns, you will have 1000x errors in timing.

2.2 Operating Conditions (PVT)

2.2.1 Theory

Operating conditions define the Process, Voltage, and Temperature (PVT) corner under which timing is analyzed. Each PVT corner has a different timing behavior:

- Process (P): Variation in transistor manufacturing — Fast (F), Typical (T), Slow (S)
- Voltage (V): Supply voltage variation — affects switching speed
- Temperature (T): Operating temperature — hot = slower CMOS, faster at cryogenic

2.2.2 Common PVT Corners

Corner Name	PVT	Use Case
Worst-Case Slow (WCS)	Slow, 0.9V, 125°C	Setup check (slowest path)
Best-Case Fast (BCF)	Fast, 1.1V, -40°C	Hold check (fastest path)
Typical (TYP)	Typical, 1.0V, 25°C	Functional simulation
WCCOM	Slow, 0.9V, 0°C	Commercial worst-case

2.2.3 Syntax

```
set_operating_conditions [-name <condition_name>]
                         [-library <lib_name>]
                         [-min <condition_name>]
                         [-max <condition_name>]

# Example: Use worst-case for setup, best-case for hold
set_operating_conditions -max WCCOM -min BCCOM

# Example: Single corner analysis
set_operating_conditions -name TYPICAL
```

✓**TIP** In advanced nodes (28nm and below), timing engineers typically run multiple corners. The SDC may specify different operating conditions for max (setup) and min (hold) analysis.

2.3 Wireload Models vs RC Extraction

2.3.1 Wireload Models (Pre-Layout)

Before physical layout is complete, the tool has no knowledge of actual wire lengths. Wireload models (WLMs) are statistical estimates of wire parasitics based on fanout count. They are library-provided and used during synthesis.

```
# Set a specific wireload model
set_wire_load_model -name "5K_hvratio_1_4" -library "slow"

# Set wireload mode
set_wire_load_mode top      ;# Use top-level model for all instances
set_wire_load_mode enclosed ;# Use model of smallest enclosing block
set_wire_load_mode segmented;# Different models for different blocks
```

2.3.2 RC Extraction (Post-Layout)

After place-and-route, actual wire lengths are known. The parasitic extraction tool (StarRC, Calibre xRC) generates a SPEF (Standard Parasitic Exchange Format) file. This SPEF is back-annotated into STA for accurate post-layout timing.

```
# In PrimeTime, read SPEF for accurate parasitics
read_parasitics -format SPEF design_postroute.spef

# Or use rc_script_file for automation
```

 WARNING Always use RC extraction (SPEF) for final timing sign-off. Wireload models are only acceptable for early synthesis exploration.

SECTION SUMMARY

- ✓ set_units defines time, capacitance, and other units — must be placed at top of SDC
- ✓ PVT corners define the operating environment — worst-case for setup, best-case for hold
- ✓ Wireload models are used pre-layout; SPEF with RC extraction is used for sign-off

Section 3: Clock Constraints

Clock constraints are the most critical section of any SDC file. Incorrect clock definitions cause widespread timing violations or missed violations. Every register in the design is clocked — the clock definition directly determines which paths are checked and how.

3.1 create_clock

3.1.1 Theory

The create_clock command defines a clock on a port or pin of the design. It specifies the clock period (frequency), waveform (duty cycle), and name. Every flip-flop in the fanout of this clock source will be considered clocked.

3.1.2 Why It Is Needed

Without a clock definition, the tool treats all paths as combinational and cannot perform setup/hold analysis. The clock period directly sets the timing budget for all sequential paths captured by that clock.

3.1.3 Syntax

```
create_clock
    -name <clock_name>
    -period <period_in_time_units>
    [-waveform {<rise_edge> <fall_edge>}]
    [<source_port_or_pin>]

# period: Full clock period in SDC time units (e.g., 10 = 10ns at 100MHz)
# waveform: {rise_time fall_time} — default is {0 period/2} = 50% duty cycle
```

3.1.4 Real Examples

```
# Example 1: Basic 100MHz clock (10ns period, 50% duty cycle)
create_clock -name CLK -period 10 [get_ports CLK]

# Example 2: 500MHz clock with explicit 50% waveform
create_clock -name CLK_FAST -period 2.0 -waveform {0 1.0} [get_ports CLK_FAST]

# Example 3: 200MHz clock with 40% duty cycle (0.8ns high, 2ns period)
create_clock -name CLK200 -period 5.0 -waveform {0 2.0} [get_ports CLK200]

# Example 4: Virtual clock (no physical source — for I/O timing)
create_clock -name VIRT_CLK -period 10

# Example 5: Multiple clocks
create_clock -name SYS_CLK  -period 10    [get_ports SYS_CLK]
create_clock -name FAST_CLK -period 4.0   [get_ports FAST_CLK]
create_clock -name SLOW_CLK -period 100   [get_ports SLOW_CLK]
```

3.1.5 Virtual Clocks

A virtual clock has no physical source in the design. It is used to model the timing of external interfaces when the I/O is clocked by an external device that shares the same frequency/phase but does not physically enter the chip.

```
# Virtual clock for I/O constraints modeling external device
create_clock -name VCLK -period 10
set_input_delay  -clock VCLK -max 3.0 [get_ports data_in]
set_output_delay -clock VCLK -max 3.0 [get_ports data_out]
```

 WARNING Always verify that every clock port in the design is covered by a create_clock command. Missing clocks result in unconstrained paths.

3.1.6 Common Mistakes

- Wrong period: Using MHz directly instead of converting to ns (1/freq)
- Missing clock source: Writing create_clock without [get_ports CLK] — creates a virtual clock unintentionally
- Wrong waveform: Specifying {0 10} for a 10ns clock (fall edge = period = wrong)
- Duplicate clocks: Defining same port twice causes conflicts

3.2 create_generated_clock

3.2.1 Theory

A generated clock is derived from a master clock through clock dividers, multiplexers, clock buffers, PLLs, or any combinational logic. The create_generated_clock command defines such derived clocks, maintaining a parent-child relationship with the source clock. This relationship is used by STA for proper timing checks across clock domains.

3.2.2 Why It Is Needed

Generated clocks propagate through clock buffers, dividers, and MUXes inside the chip. Without declaring them, the tool either:

- Treats post-divider paths as unconstrained, or
- Uses the original master clock, which is incorrect for divided-down domains

3.2.3 Syntax

```
create_generated_clock
    -name <gen_clk_name>
    -source <master_clock_pin>
    [-divide_by <integer>]
    [-multiply_by <integer>]
    [-edges {<edge_list>}]
    [-edge_shift {<shift_list>}]
    [-invert]
    [-duty_cycle <percent>]
    [-combinational]
    [<target_pin_or_port>]
```

3.2.4 Real Examples

```
# Example 1: Divided-by-2 clock from flip-flop output
# Master: 200MHz (5ns period) on CLK port
create_clock -name MASTER -period 5.0 [get_ports CLK]

# Generated clock: 100MHz on Q pin of clock divider FF
create_generated_clock -name CLK_DIV2 \
    -source [get_ports CLK] \
    -divide_by 2 \
    [get_pins clk_divider/Q]

# Example 2: Generated clock through a MUX
create_generated_clock -name CLK_MUX_0 \
    -source [get_pins pll/CLKOUT] \
    -divide_by 1 \
    [get_pins clk_mux/Y]

# Example 3: Inverted clock
create_generated_clock -name CLK_INV \
    -source [get_ports CLK] \
    -invert \
```

```
    [get_pins inv_buf/Y]

# Example 4: Edge-based generated clock (specify edges from master)
# Master edges: 0, 5, 10, 15, 20... (200MHz)
# Take edges 1, 3, 5 from master -> creates a 100MHz derived clock
create_generated_clock -name CLK_EDGE \
    -source [get_ports CLK] \
    -edges {1 3 5} \
    [get_pins clk_gen/Q]
```

✓**TIP** PLL output clocks should always be modeled with create_generated_clock. The -source should be the PLL reference input, not the output.

3.2.5 Handling Clock MUX Scenarios

When a clock MUX selects between two clocks, you should define a generated clock on the MUX output for each input condition:

```
# Clock MUX: select=0 uses CLK_A, select=1 uses CLK_B
create_generated_clock -name MUX_CLK_A \
    -source [get_ports CLK_A] \
    -divide_by 1 \
    [get_pins clk_mux/Y]

create_generated_clock -name MUX_CLK_B \
    -source [get_ports CLK_B] \
    -divide_by 1 \
    -add \
    [get_pins clk_mux/Y]

# -add flag is required when multiple clocks exist on the same pin
```

3.3 Clock Latency

3.3.1 Theory

Clock latency is the delay from the clock source to the clock pins of flip-flops. It has two components:

- Source Latency (insertion delay): Delay from the external clock source to the clock definition point (e.g., clock port of the chip)
- Network Latency: Delay from the clock port through the internal clock tree to the flip-flop clock pins

3.3.2 Why It Is Needed

Before clock tree synthesis (CTS), the clock tree is ideal (zero delay). Source latency models the external board-level clock delay. Network latency models the estimated internal clock tree delay for pre-CTS timing analysis.

3.3.3 Syntax

```
set_clock_latency
    [-rise] [-fall]
    [-min] [-max]
    [-source]      ;# specifies source latency
    <latency_value>
    <clock_object>
```

3.3.4 Real Examples

```
# Source latency: Clock arrives 2ns late from the board
set_clock_latency -source 2.0 [get_clocks SYS_CLK]

# Network latency: Estimated clock tree delay of 1.5ns (pre-CTS only)
set_clock_latency 1.5 [get_clocks SYS_CLK]
```

```
# Min/max latency for hold/setup respectively
set_clock_latency -source -min 1.8 [get_clocks SYS_CLK]
set_clock_latency -source -max 2.2 [get_clocks SYS_CLK]
```

 WARNING After CTS (clock tree synthesis), remove set_clock_latency for network latency — propagated clock mode will use actual clock tree delays from SPEF. Keep source latency as it models board-level delay.

3.4 Clock Uncertainty

3.4.1 Theory

Clock uncertainty accounts for variations in clock arrival times that are difficult to predict precisely. It includes:

- Jitter: Cycle-to-cycle variation in clock period due to PLL noise, power supply noise
- Skew: Difference in clock arrival times at different flip-flops due to clock tree imbalance
- Margin: Guard-band added for safety

3.4.2 Setup vs Hold Uncertainty

Clock uncertainty applies to both setup and hold checks, but with different effects:

- Setup uncertainty: Tightens the setup check by reducing available timing budget
- Hold uncertainty: Tightens the hold check by reducing the minimum delay margin

3.4.3 Syntax

```
set_clock_uncertainty
    [-setup]
    [-hold]
    [-from <clock>]
    [-to <clock>]
    <uncertainty_value>
    [<clock_object>]
```

3.4.4 Real Examples

```
# General uncertainty on all paths of a clock
set_clock_uncertainty 0.15 [get_clocks SYS_CLK]

# Separate setup and hold uncertainty
set_clock_uncertainty -setup 0.15 [get_clocks SYS_CLK]
set_clock_uncertainty -hold  0.05 [get_clocks SYS_CLK]

# Cross-clock domain uncertainty (from CLK_A to CLK_B)
set_clock_uncertainty -from [get_clocks CLK_A] \
                      -to   [get_clocks CLK_B] \
                      0.3

# Typical industry values:
# Pre-CTS: 0.5ns - 1.0ns (large guard-band for clock tree uncertainty)
# Post-CTS: 0.1ns - 0.2ns (actual jitter + residual skew)
```

3.4.5 Industry-Standard Uncertainty Values

Design Phase	Setup Uncertainty	Hold Uncertainty
Pre-CTS (Synthesis)	0.5 - 1.0 ns	0.2 - 0.5 ns
Post-CTS (STA)	0.1 - 0.2 ns	0.05 - 0.1 ns
Advanced Node (7nm)	0.05 - 0.1 ns	0.02 - 0.05 ns

3.5 Clock Transition

3.5.1 Theory

Clock transition (slew) is the rise and fall time of the clock signal. An ideal clock has zero transition time. In reality, the clock signal has finite slew rates. Specifying clock transition helps the tool compute accurate setup/hold timing since cell delays are slew-dependent.

3.5.2 Syntax and Example

```
set_clock_transition
    [-rise] [-fall]
    [-min] [-max]
    <transition_value>
    <clock_objects>

# Example: Set clock transition for pre-CTS analysis
set_clock_transition 0.1 [get_clocks SYS_CLK]

# Separate rise/fall transitions
set_clock_transition -rise 0.08 [get_clocks SYS_CLK]
set_clock_transition -fall 0.10 [get_clocks SYS_CLK]
```

✓**TIP** Post-CTS, remove set_clock_transition — the actual clock tree waveform captured in SPEF will be used instead.

3.6 Ideal vs Propagated Clocks

3.6.1 Ideal Clocks (Pre-CTS)

By default, clocks in SDC are ideal — they have zero latency and zero transition time through the clock tree. This is the correct mode for synthesis and pre-CTS STA. Clock uncertainty and latency are applied manually to model the expected clock tree behavior.

3.6.2 Propagated Clocks (Post-CTS)

After clock tree synthesis, the set_propagated_clock command tells the STA tool to compute actual clock arrival times by propagating the clock through the physical clock tree network. Parasitics from SPEF are used for accurate delay computation.

```
# Switch to propagated clock mode after CTS
set_propagated_clock [get_clocks SYS_CLK]

# Or apply to all clocks
set_propagated_clock [all_clocks]
```

3.6.3 Comparison Table

Property	Ideal Clock	Propagated Clock
Clock tree delay	Zero (or manual latency)	Actual from SPEF
Clock skew	Controlled by uncertainty	Actual from CTS
When used	Pre-CTS, synthesis	Post-CTS, sign-off
Accuracy	Approximate	High accuracy

3.7 Clock Gating Considerations

3.7.1 Theory

Clock gating is a power reduction technique where a combinational gate (AND/OR) sits in the clock path to disable the clock when not needed. SDC must account for the extra delay introduced by the gating cell and ensure proper timing at the gating cell's enable pin.

3.7.2 Clock Gating Check

The STA tool automatically performs clock gating checks if the design uses ICG (Integrated Clock Gating) cells. The enable signal must be stable before the clock edge — this is checked like a setup/hold check on the enable pin.

```
# Allow clock gating checks (usually default in modern tools)
set_clock_gating_check -setup 0.0 -hold 0.0 [get_cells clk_gate_u0]

# Specify a margin on the enable signal
set_clock_gating_check -setup 0.2 [all_registers -clock_gating_pins]
```

 WARNING Modern ICG cells (latch-based gate) have built-in glitch protection. The enable must be stable during the latch transparent window — violating this causes glitches and potential data corruption.

SECTION SUMMARY

- ✓ create_clock defines the master clock — period, waveform, source
- ✓ create_generated_clock defines derived clocks from dividers, MUXes, PLLs
- ✓ Clock latency models source (board) and network (internal tree) delays
- ✓ Clock uncertainty accounts for jitter and skew — higher pre-CTS, lower post-CTS
- ✓ set_propagated_clock enables accurate post-CTS timing using actual clock tree parasitics
- ✓ Clock gating must be properly constrained to avoid enable glitches

Section 4: Input Constraints

Input constraints define the timing environment seen at the input ports of the design. They model the external circuit that drives the inputs — specifically, how late the input data arrives relative to the clock, and what the signal quality looks like.

4.1 set_input_delay

4.1.1 Theory

set_input_delay specifies the time from the active clock edge to when the input data is valid (arrives at the input port). The STA tool uses this to determine how much of the clock period remains for the internal path from the input port to the first register.

4.1.2 Path Budget Equation

```
Available Internal Timing Budget:

  Tinternal = Tclock_period - Tinput_delay - Tsetup_register

Where:
  Tclock_period = Clock period (e.g., 10ns)
  Tinput_delay  = set_input_delay value (e.g., 3ns)
  Tsetup_reg    = Setup time of the capturing flip-flop

If Tinput_delay = 3ns and period = 10ns:
  Internal path must complete in < (10 - 3) = 7ns
```

4.1.3 Syntax

```
set_input_delay
    [-clock <clock_name>]
    [-max]   ;# Setup check (default)
    [-min]   ;# Hold check
    [-clock_fall]    ;# Data launched on falling edge of clock
    [-add_delay]     ;# Add to existing delay (multiple clocks on same port)
    <delay_value>
    <port_list>
```

4.1.4 Real Examples

```
# Example 1: Basic input delay — data valid 3ns after rising clock edge
set_input_delay -clock SYS_CLK -max 3.0 [get_ports data_in]
set_input_delay -clock SYS_CLK -min 0.5 [get_ports data_in]

# Example 2: Multiple input ports with same delay
set_input_delay -clock SYS_CLK -max 4.0 [get_ports {addr[*] data[*]}]

# Example 3: Data launched on falling edge of external clock
set_input_delay -clock SYS_CLK -clock_fall -max 3.5 [get_ports ddr_data_in]

# Example 4: Input relative to a virtual clock
create_clock -name VCLK -period 10
set_input_delay -clock VCLK -max 2.0 [get_ports {ctrl_in[*]}]

# Example 5: Constrain all inputs except clock and reset
set all_inputs_except_clk [remove_from_collection \
    [all_inputs] \
    [get_ports {CLK RESET_N}]]
set_input_delay -clock SYS_CLK -max 3.0 $all_inputs_except_clk
```

 WARNING Always specify both -max (for setup) and -min (for hold) input delays. Specifying only -max leaves hold paths unconstrained.

4.2 set_input_transition

4.2.1 Theory

set_input_transition specifies the slew rate (transition time) of the input signal arriving at the chip boundary. Cell delays inside the design depend on the input slew — a slower slew means slower cells and longer delays.

4.2.2 Syntax and Example

```
set_input_transition
    [-rise] [-fall]
    [-min] [-max]
    <transition_value>
    <port_list>

# Example: 0.2ns input transition on all data inputs
set_input_transition 0.2 [get_ports {data_in[*]}]

# Example: Separate rise/fall transitions
set_input_transition -rise 0.15 [get_ports {addr[*]}]
set_input_transition -fall 0.20 [get_ports {addr[*]}]
```

4.3 set_driving_cell

4.3.1 Theory

set_driving_cell is a more accurate alternative to set_input_transition. Instead of specifying a transition value directly, you specify the actual cell that drives the input port. The tool then uses the cell's drive strength and the load on the wire to calculate the actual input transition. This is more physically accurate.

4.3.2 Syntax and Example

```
set_driving_cell
    -lib_cell <cell_name>
    [-library <lib_name>]
    [-pin <output_pin>]
    [-from_pin <input_pin>]
    [-input_transition_rise <value>]
    [-input_transition_fall <value>]
    [-dont_size]
    <port_list>

# Example: Drive all inputs with a standard drive-4 buffer
set_driving_cell -lib_cell BUF4X [get_ports {data_in[*]}]

# Example: More detailed driving cell specification
set_driving_cell \
    -lib_cell INVX8 \
    -library slow_1p0v_125c \
    -pin Y \
    [get_ports clk_in]
```

4.3.3 set_driving_cell vs set_input_transition

set_driving_cell	set_input_transition
More accurate — uses cell drive strength	Less accurate — simple transition value

Accounts for load on the wire	Fixed slew regardless of load
Preferred for final synthesis/sign-off	Acceptable for early exploration
Requires a library cell name	Only needs a number

SECTION SUMMARY

✓ set_input_delay models when external data arrives relative to the clock — critical for setup/hold on input paths
✓ Always specify both -max and -min input delays
✓ set_input_transition models the slew of the external driver signal
✓ set_driving_cell is more accurate — preferred for synthesis and sign-off

Section 5: Output Constraints

Output constraints define the timing requirements at the output ports of the design. They model the external circuit that captures the output — how much time the external device needs to capture the data, and what load it presents to the driver.

5.1 set_output_delay

5.1.1 Theory

set_output_delay specifies the time before the next clock edge that the output data must be stable at the output port. The tool checks that the internal path from the launching flip-flop to the output port completes in time to meet this requirement.

5.1.2 Path Budget Equation

```
Maximum internal path delay from FF to output port:

  Tpath_max = Tclock_period - Toutput_delay

  If period = 10ns, output_delay = 2ns:
  Internal path (FF Q to port) must be < 8ns
```

5.1.3 Syntax

```
set_output_delay
    [-clock <clock_name>]
    [-max]   ;# Setup check (default)
    [-min]   ;# Hold check
    [-clock_fall]
    [-add_delay]
    <delay_value>
    <port_list>
```

5.1.4 Real Examples

```
# Example 1: Output must be stable 2ns before next clock edge
set_output_delay -clock SYS_CLK -max 2.0 [get_ports data_out]
set_output_delay -clock SYS_CLK -min 0.2 [get_ports data_out]

# Example 2: All output ports with same constraint
set_output_delay -clock SYS_CLK -max 3.0 [get_ports {result[*] valid done}]

# Example 3: Double data rate output (falling edge capture)
set_output_delay -clock SYS_CLK -clock_fall -max 1.5 [get_ports ddr_out]
set_output_delay -clock SYS_CLK -max 1.5 -add_delay [get_ports ddr_out]
```

5.2 set_load

5.2.1 Theory

set_load specifies the capacitive load at the output ports. This models the external capacitance that the output driver must charge/discharge. The load affects the transition time of the output, which in turn affects the delay of the last cell in the output path.

5.2.2 Syntax and Examples

```
set_load [-min] [-max] [-subtract_pin_load] [-pin_load]
         <load_value> <port_list>

# Example 1: 5pF load on all output ports
```

```
set_load 5.0 [all_outputs]

# Example 2: Different loads for different ports
set_load 2.0  [get_ports data_out[*]]
set_load 10.0 [get_ports high_drive_out]

# Example 3: Load in fF (if units set to fF)
set_units -capacitance fF
set_load 50 [all_outputs]  ;# 50fF
```

✓**TIP** set_load is critical for correct output transition and delay computation. Underestimating the load leads to optimistic timing; overestimating wastes area with unnecessarily large drivers.

5.2.3 Output Interface Timing — Complete Example

```
# Complete output interface constraint for a synchronous bus
# External device needs data 3ns before clock edge (setup = 2ns, board delay = 1ns)
# External device hold requirement = 0.5ns after clock edge

create_clock -name SYS_CLK -period 10 [get_ports SYS_CLK]

set_output_delay -clock SYS_CLK -max  3.0 [get_ports {out_data[*] out_valid}]
set_output_delay -clock SYS_CLK -min -0.5 [get_ports {out_data[*] out_valid}]
set_load 4.0 [get_ports {out_data[*] out_valid}]
```

SECTION SUMMARY

- ✓ set_output_delay models external device setup requirements at the output port
- ✓ Both -max (setup) and -min (hold) must be specified for complete constraint coverage
- ✓ set_load models the external capacitive load — affects output transition and delay accuracy

Section 6: Timing Exceptions

Timing exceptions tell the STA tool to modify or override the default timing analysis for specific paths. These are among the most powerful — and dangerous — SDC commands. Incorrect exceptions can hide real timing problems or waste resources on non-critical paths.

6.1 set_false_path

6.1.1 Theory

A false path is a timing path that exists in the gate-level netlist but does not represent a real functional timing requirement. The STA tool should not check timing on false paths. However, the path still physically exists — only the timing constraint is removed.

6.1.2 When to Use set_false_path

- Asynchronous resets or sets that are only active during non-functional conditions
- Test logic paths (scan enable, BIST control) that are inactive in functional mode
- Clock domain crossings where no real data transfer occurs
- Paths between mutually exclusive operating modes
- Path from output of a clock mux to other logic (not clocked by that path)

6.1.3 Syntax

```
set_false_path
    [-setup]
    [-hold]
    [-from <start_point>]
    [-to <end_point>]
    [-through <through_point>]
    [-rise_from / -fall_from]
    [-rise_to / -fall_to]
```

6.1.4 Real Examples

```
# Example 1: Asynchronous reset path — active only at power-up, not in function
set_false_path -from [get_ports ASYNC_RESET_N]

# Example 2: Test mode path (scan_enable only active during test)
set_false_path -from [get_ports SCAN_EN]

# Example 3: False path between two asynchronous clock domains
set_false_path -from [get_clocks CLK_A] -to [get_clocks CLK_B]

# Example 4: False path through a specific cell (test mux)
set_false_path -through [get_pins test_mux/S]

# Example 5: One-directional false path (A->B false, B->A still checked)
set_false_path -from [get_clocks FAST_CLK] -to [get_clocks SLOW_CLK]

# Example 6: Setup-only false path (hold still checked)
set_false_path -setup -from [get_clocks CLK_A] -to [get_clocks CLK_B]
```

❌**CAUTION** NEVER use set_false_path as a shortcut to 'fix' real timing violations. False paths silence violations, but if the path is real, you will get silicon failures.

6.1.5 Common Mistakes with set_false_path

- Making entire clock domain crossing false when some paths are real — use set_clock_groups instead

- False path from an asynchronous input that actually feeds synchronous logic
- Using -through carelessly — can match unintended paths

6.2 set_multicycle_path

6.2.1 Theory

A multicycle path is a timing path where the data takes more than one clock cycle to propagate. By default, STA assumes all paths must complete in one clock period. If a designer intentionally uses multiple cycles (e.g., a slow control signal or a pipelined divider), the constraint must be relaxed accordingly.

6.2.2 Multicycle Path — Setup Analysis

When you specify a multicycle path of N cycles for setup, the STA tool moves the required time edge N cycles forward:

```
# Default setup check: data launched at cycle 0 must arrive by cycle 1
# MCP 2 for setup: data launched at cycle 0 can arrive at cycle 2

set_multicycle_path -setup 2 -from [get_cells slow_reg] -to [get_cells capture_reg]

# IMPORTANT: When setting MCP for setup, you MUST also set hold to N-1
# to prevent the hold check from becoming too tight
set_multicycle_path -hold 1 -from [get_cells slow_reg] -to [get_cells capture_reg]
```

6.2.3 Why Hold Must Be Adjusted

The hold check default is 0 cycles. When MCP setup is set to N, the hold check (without adjustment) would be N-1 cycles back, which is extremely tight. The hold must be set to N-1 to re-align the hold check to the correct edge.

```
# MCP Setup = 2, Hold must = 1
set_multicycle_path -setup 2 -from FF_A -to FF_B
set_multicycle_path -hold  1 -from FF_A -to FF_B

# MCP Setup = 3, Hold must = 2
set_multicycle_path -setup 3 -from FF_A -to FF_B
set_multicycle_path -hold  2 -from FF_A -to FF_B
```

6.2.4 Multicycle Path — Real Industry Scenarios

Scenario	MCP Setting
Slow configuration register (updated every N cycles)	setup N, hold N-1
Integer divider with known latency of 2 cycles	setup 2, hold 1
Memory interface with 2-cycle latency	setup 2, hold 1
Low-frequency control path in high-speed design	setup depends on ratio

6.2.5 Real Examples

```
# Example 1: 2-cycle multicycle path — data valid every 2 cycles
set_multicycle_path -setup 2 \
    -from [get_clocks SYS_CLK] \
    -through [get_pins slow_adder/out] \
    -to [get_clocks SYS_CLK]
set_multicycle_path -hold 1 \
    -from [get_clocks SYS_CLK] \
```

```
    -through [get_pins slow_adder/out] \
    -to [get_clocks SYS_CLK]

# Example 2: Configuration register — updated once every 8 cycles
set_multicycle_path -setup 8 -from [get_cells cfg_reg[*]] -to [get_cells data_path[*]]
set_multicycle_path -hold  7 -from [get_cells cfg_reg[*]] -to [get_cells data_path[*]]

# Example 3: Cross-frequency MCP (CLK_A at 100MHz, CLK_B at 50MHz)
# Data at CLK_A needs to be captured at CLK_B — 2:1 ratio
set_multicycle_path -setup 2 -from [get_clocks CLK_A] -to [get_clocks CLK_B]
set_multicycle_path -hold  1 -from [get_clocks CLK_A] -to [get_clocks CLK_B]
```

 WARNING Always verify multicycle paths with simulation. The hardware must guarantee that data is truly valid at the required edge. MCP does not add hardware synchronization — it only relaxes the timing check.

6.3 set_max_delay / set_min_delay

6.3.1 set_max_delay Theory

set_max_delay overrides the default setup timing requirement on a path. Instead of using the clock period as the required time, the tool uses the specified maximum delay value. This is useful for paths that must complete faster than the clock period (data valid windows) or for defining point-to-point timing budgets.

```
set_max_delay
    [-datapath_only]  ;# CRITICAL: bypasses clock period, ignores setup uncertainty
    [-ignore_clock_latency]
    <delay_value>
    [-from <start>] [-to <end>] [-through <through>]

# Example 1: CDC path — data must arrive within 3ns (but no clocked check)
set_max_delay -datapath_only 3.0 \
    -from [get_clocks SYS_CLK] \
    -to [get_clocks FAST_CLK]

# Example 2: Combinational output path with tight budget
set_max_delay 5.0 -from [get_ports in_data] -to [get_ports out_comb]
```

6.3.2 set_min_delay Theory

set_min_delay overrides the default hold timing requirement. It specifies the minimum required delay on a path — data must arrive no earlier than this value after the reference clock edge.

```
set_min_delay <delay_value>
    [-from <start>] [-to <end>] [-through <through>]

# Example: Output must not change for at least 0.5ns after clock edge (hold)
set_min_delay 0.5 -to [get_ports {data_out[*]}]
```

6.3.3 -datapath_only Flag — Critical Usage

When -datapath_only is used with set_max_delay, the clock period check is completely replaced by the specified value. No clock uncertainty is added. This is commonly used for asynchronous CDC paths where you want to budget the combinational path delay without any clock relationship.

 WARNING -datapath_only disables the setup check's clock uncertainty. Use with care — it removes a safety margin. Also, hold check is still computed based on clock unless set_min_delay is also applied.

6.4 When to Use and When NOT to Use

6.4.1 Decision Guide

Scenario	Correct Command	Wrong Approach

Async reset (active only at startup)	set_false_path	set_multicycle_path
2-cycle pipeline path	set_multicycle_path -setup 2	set_false_path
Async CDC (no data relationship)	set_clock_groups -async	set_false_path -from clk -to clk
Test scan path (scan mode only)	set_false_path -from SCAN_EN	Do nothing (leave constrained)
Cross-domain with FIFO synchronizer	set_max_delay -datapath_only	set_false_path

SECTION SUMMARY

✓ set_false_path: Remove timing check on paths that are functionally irrelevant
✓ set_multicycle_path: Relax timing for paths that take N clock cycles — ALWAYS adjust hold = setup - 1
✓ set_max_delay: Override setup check with a specific delay budget
✓ set_min_delay: Override hold check with a specific minimum delay
✓ NEVER use exceptions to hide real violations — they mask actual silicon bugs

Section 7: Clock Domain Crossing (CDC)

Clock Domain Crossing (CDC) occurs when a signal generated in one clock domain is captured in another clock domain. CDC is a major source of metastability and silicon bugs if not handled correctly. SDC must properly define clock domain relationships to guide both STA and synthesis.

7.1 set_clock_groups

7.1.1 Theory

set_clock_groups defines the relationship between multiple clocks. The most common use is to declare clocks as asynchronous — meaning STA should not perform timing checks between them. This is the preferred way to handle true CDC paths over set_false_path.

7.1.2 Why set_clock_groups Over set_false_path for CDC

While set_false_path -from CLK_A -to CLK_B would achieve a similar effect for one direction, set_clock_groups is more maintainable and clearly communicates design intent. It also handles both directions simultaneously and is recognized by CDC verification tools.

7.1.3 Syntax

```
set_clock_groups
    -asynchronous            ;# Most common: clocks are async, no STA checks
    -physically_exclusive    ;# Clocks are mutually exclusive (only one active)
    -logically_exclusive     ;# Logically exclusive but may exist simultaneously
    -allow_paths             ;# Allow timing analysis even if in different groups
    -group {<clock_list>}    ;# Specify multiple groups
    [-name <group_name>]
```

7.1.4 Real Examples

```
# Example 1: Two completely asynchronous clocks — no timing checks between them
set_clock_groups -asynchronous \
    -group {SYS_CLK} \
    -group {USB_CLK}

# Example 2: Multiple asynchronous domains
set_clock_groups -asynchronous \
    -group {SYS_CLK CLK_DIV2} \
    -group {PCIE_CLK} \
    -group {DDR_CLK}

# Example 3: Physically exclusive clocks (MUX-selected, only one active)
set_clock_groups -physically_exclusive \
    -group {CLK_HIGH_SPEED} \
    -group {CLK_LOW_SPEED}

# Example 4: Logically exclusive functional modes
set_clock_groups -logically_exclusive \
    -group {FUNC_CLK} \
    -group {TEST_CLK}
```

7.2 Asynchronous Clock Basics and CDC Handling

7.2.1 What is Metastability?

When a flip-flop's setup or hold time is violated, its output can enter a metastable state — an intermediate voltage level between 0 and 1. Given enough time (mean time between failures — MTBF), the FF resolves to a valid state. CDC synchronizers are designed to give the FF enough resolution time.

7.2.2 CDC Synchronization Architectures

```
// 2-FF Synchronizer (most common single-bit CDC)
// Place two back-to-back flip-flops in the destination domain

module sync_2ff (
    input  d_src,   // Signal from source domain
    input  clk_dst, // Destination domain clock
    output d_dst    // Synchronized output
);
    reg ff1, ff2;
    always @(posedge clk_dst) begin
        ff1 <= d_src;  // First FF: may go metastable
        ff2 <= ff1;    // Second FF: resolves metastability
    end
    assign d_dst = ff2;
endmodule

// In SDC, the path from d_src to ff1.D should be:
set_max_delay -datapath_only 1.5 -from [get_cells src_ff] \
    -to [get_cells sync_2ff/ff1]
```

7.2.3 SDC for CDC Paths

```
# Step 1: Declare all async clock pairs
set_clock_groups -asynchronous \
    -group {CLK_SRC} \
    -group {CLK_DST}

# Step 2: Apply datapath-only budget on synchronizer input path
# (This constrains the combinational logic before the synchronizer FF)
set_max_delay -datapath_only [expr {$clk_dst_period * 0.8}] \
    -from [get_clocks CLK_SRC] \
    -to [get_cells {*sync_ff*}]
```

✅**TIP** CDC analysis tools (Synopsys VC SpyGlass CDC, Mentor Questa CDC) should be used in addition to SDC to verify synchronizer implementation and identify all CDC crossings.

SECTION SUMMARY

- ✓ set_clock_groups -asynchronous is the preferred way to declare async clock domains
- ✓ CDC paths should use 2-FF synchronizers for single-bit signals; Gray code for buses
- ✓ set_max_delay -datapath_only is used to budget the logic before the synchronizer input
- ✓ Always validate CDC coverage with dedicated CDC analysis tools

Section 8: Design Rule Constraints (DRC)

Design Rule Constraints (DRCs) define physical and electrical limits that the synthesized or routed design must not exceed. These constraints ensure that the design is manufacturable and operates correctly from an electrical standpoint.

8.1 set_max_transition

8.1.1 Theory

Transition time is the time a signal takes to change from one logic level to another (slew rate). Excessively slow transitions:

- Increase cell propagation delay
- Increase short-circuit current (both P and N transistors conduct simultaneously)
- Lead to crowbar current and power/reliability issues
- Can cause gate oxide stress in advanced nodes

8.1.2 Syntax and Examples

```
set_max_transition <transition_value> <design_or_net>

# Set 0.5ns maximum transition on entire design
set_max_transition 0.5 [current_design]

# Stricter limit on clock network
set_max_transition 0.2 [get_clocks SYS_CLK]

# Even stricter on a specific module
set_max_transition 0.3 [get_cells fast_module]
```

8.2 set_max_capacitance

8.2.1 Theory

Maximum capacitance limits the total load a cell output must drive. Driving too much capacitance:

- Slows down the cell (increases delay)
- Increases the transition time (violates max_transition)
- Increases dynamic power consumption

```
set_max_capacitance <cap_value> <design_or_net>

# Set 0.5pF maximum capacitance on all nets
set_max_capacitance 0.5 [current_design]

# Stricter limit on a specific bus
set_max_capacitance 0.3 [get_nets {data_bus[*]}]
```

8.3 set_max_fanout

8.3.1 Theory

Fanout is the number of inputs a single gate output drives. High fanout:

- Increases capacitive load and transition time
- Increases vulnerability to noise
- Can cause maximum capacitance and transition violations

```
set_max_fanout <fanout_value> <design_or_port>
```

```
# Maximum 32 loads on any gate output
set_max_fanout 32 [current_design]

# Tighter control on control signals
set_max_fanout 8 [get_ports {enable ctrl_*}]
```

8.3.2 Impact on Synthesis

DRC Type	How Synthesis Fixes It
Max transition violation	Inserts buffers to improve slew
Max capacitance violation	Inserts buffers to reduce load per driver
Max fanout violation	Clones gates (buffer tree) to split fanout

 WARNING DRC constraints are physical design requirements. Violating them can cause timing to be inaccurate (cells characterized at low transition only) and reliability issues at silicon.

SECTION SUMMARY

✓ set_max_transition limits signal slew rate — prevents power and reliability issues
✓ set_max_capacitance limits load per driver — maintains speed and transition quality
✓ set_max_fanout limits number of loads — prevents high-fanout degradation
✓ All three DRCs drive buffer insertion and gate cloning during synthesis

Section 9: Advanced Constraints

9.1 set_case_analysis

9.1.1 Theory

set_case_analysis forces a constant logic value (0 or 1) on a specific pin or port for the purposes of timing analysis. This is used to model operating modes, MUX selection, or functional enable signals that are known to have a fixed value in a particular operating scenario.

9.1.2 Syntax and Examples

```
set_case_analysis <value> <object>
# value: 0, 1, rising, falling, zero, one

# Example 1: Test mode disabled (scan_en = 0 in functional mode)
set_case_analysis 0 [get_ports SCAN_EN]

# Example 2: MUX select forces one path to be analyzed
set_case_analysis 1 [get_pins clk_mux/SEL]

# Example 3: Power gating header cell always on during timing
set_case_analysis 1 [get_pins power_switch/ENAB]
```

✓**TIP** set_case_analysis only affects timing analysis — it does not modify the actual netlist. Use it to model scenarios accurately without creating multiple netlists.

9.2 set_disable_timing

9.2.1 Theory

set_disable_timing removes a timing arc from a cell during STA. This is different from set_false_path (which removes a path) — set_disable_timing removes the arc at the cell level, affecting all paths through that arc.

```
set_disable_timing [-from <pin>] [-to <pin>] <object>

# Example: Disable timing arc from CLK to Q (model a non-clocked path)
set_disable_timing -from CK -to Q [get_cells special_latch]

# Example: Disable timing on a specific net
set_disable_timing [get_nets test_net]
```

9.3 Path Grouping

9.3.1 Theory

Path groups organize timing paths into named categories. The STA tool reports timing for each group separately. This helps identify which type of paths (register-to-register, input-to-register, etc.) are the most critical.

```
group_path -name <group_name>
           [-from <start>]
           [-to <end>]
           [-through <through>]
           [-weight <value>]  ;# Higher weight = higher optimization priority

# Example: Group critical data paths
group_path -name CRITICAL_DATA \
    -from [get_clocks SYS_CLK] \
    -to [get_clocks SYS_CLK] \
```

```
    -weight 2.0

# Example: Separate input/output path groups
group_path -name INPUTS  -from [all_inputs]
group_path -name OUTPUTS -to   [all_outputs]
group_path -name REG2REG -from [all_registers] -to [all_registers]
```

9.4 Hierarchical Constraints

9.4.1 Theory

In hierarchical designs, SDC constraints can be applied at different levels of the hierarchy. The top-level SDC typically constrains the interface, while block-level SDCs constrain internal paths. Proper hierarchical constraint management prevents over-constraining at block level while maintaining correct system-level timing.

```
# Top-level SDC — constrains the full chip
create_clock -name SYS_CLK -period 10 [get_ports SYS_CLK]

# Block-level constraint — create a separate SDC for the block
# In design_compiler for the block:
current_design CORE_BLOCK
create_clock -name SYS_CLK -period 10 [get_pins clk_port]

# At top level, source the block-level constraints
source block_a.sdc
source block_b.sdc

# Or use set_hierarchy_separator if needed
set_hierarchy_separator /
```

SECTION SUMMARY

- ✓ set_case_analysis models fixed logic values for operating mode analysis
- ✓ set_disable_timing removes specific cell timing arcs
- ✓ group_path organizes path reporting and drives optimization priority
- ✓ Hierarchical SDC management separates block-level and top-level constraints

Section 10: Variations and Derates (OCV/AOCV/POCV)

In advanced semiconductor nodes (28nm, 16nm, 7nm, and below), process variations cause significant timing uncertainty. The same cell on the same chip may have different delays on different die corners or different locations. Timing derates model this variation.

10.1 Why Variation Matters

In older nodes (180nm, 90nm), process variation was relatively uniform. In advanced nodes:

- Within-die variation (WID) is significant — two identical cells placed 1mm apart may have 5-10% delay difference
- Die-to-die variation between chips further adds uncertainty
- Voltage droop, IR drop, and thermal gradients contribute additional variation

10.2 OCV (On-Chip Variation)

10.2.1 Theory

OCV applies a global derate factor to all cells. Launch path cells are slowed down (pessimistic) for setup checks, and capture path cells are sped up (optimistic). This creates a worst-case scenario for both setup and hold.

```
set_timing_derate [-cell_delay] [-net_delay]
                  [-data]       [-clock]
                  [-late]       [-early]
                  <derate_factor>

# OCV: 5% derate on all paths
# Late (setup): cells are 5% slower
set_timing_derate -late  1.05 [current_design]
# Early (hold): cells are 5% faster
set_timing_derate -early 0.95 [current_design]

# Separate clock and data derates
set_timing_derate -late  -clock 1.03 [current_design]  ;# clock 3% slower
set_timing_derate -late  -data  1.05 [current_design]  ;# data  5% slower
set_timing_derate -early -clock 0.97 [current_design]
set_timing_derate -early -data  0.95 [current_design]
```

10.3 AOCV (Advanced OCV)

10.3.1 Theory

AOCV improves upon OCV by applying path-depth-specific derates. Longer paths (more stages) tend to have variations cancel out (statistical averaging), so a smaller derate is appropriate. Short paths (few stages) may be dominated by a single cell's variation, so a larger derate applies.

AOCV tables are provided by the library vendor and contain derate values indexed by path stage count. This results in less pessimism than flat OCV, enabling better timing QoR.

```
# AOCV is typically enabled by setting the derate table
# In PrimeTime, use read_aocvm_table to load AOCV tables
# The actual command varies by tool version

# Example AOCV derate concept (not actual syntax):
# Stage 1: late_derate = 1.10 (10%)
# Stage 5: late_derate = 1.06 (6%)
# Stage 10: late_derate = 1.04 (4%)
# Stage 20+: late_derate = 1.02 (2%)
```

10.4 POCV (Parametric OCV)

10.4.1 Theory

POCV is the most advanced and accurate variation methodology. Instead of using worst/best-case corners, POCV uses statistical distributions (mean and sigma) for each cell's delay. The timing tool computes the probability distribution of path delays and applies a confidence-level-based guard band.

POCV is characterized by cell sigma values provided in the liberty (.lib) files under specific extensions. It reduces over-pessimism of OCV/AOCV while maintaining the required statistical confidence (typically 3-sigma).

```
# Enable POCV in PrimeTime
set timing_pocvm_enable true

# Set confidence level (3-sigma = 99.73% coverage)
set_pocvm_sigma_factor 3.0

# POCV derates are computed automatically from library sigma values
```

10.5 OCV Methodology Comparison

Methodology	Accuracy	When Used
OCV	Low — conservative flat derate	Older nodes, initial signoff
AOCV	Medium — depth-aware derate	28nm - 16nm nodes
POCV	High — statistical distribution	16nm and below, advanced nodes

SECTION SUMMARY

- ✓ OCV applies global derate factors to model within-die variation
- ✓ AOCV improves OCV with path-depth-aware derates — less pessimism
- ✓ POCV uses statistical cell delay distributions — most accurate, least pessimistic
- ✓ Always use the OCV methodology recommended by the foundry for the target process node

Section 11: Pre-Layout vs Post-Layout Constraints

SDC constraints evolve throughout the design flow. The constraints used during synthesis (pre-layout) differ from those used during post-route static timing analysis (post-layout). Understanding these differences is critical to avoiding sign-off surprises.

11.1 Pre-Layout Constraints (Synthesis Phase)

11.1.1 Characteristics

- Clocks are ideal (set_ideal_clock or default behavior)
- Clock latency modeled with set_clock_latency
- Clock uncertainty modeled with set_clock_uncertainty (typically 0.5 - 1.0ns)
- Clock transition specified with set_clock_transition
- Wireload models used for wire parasitics
- Input transitions specified with set_input_transition or set_driving_cell

11.1.2 Pre-Layout SDC Setup

```
# ============================================================
# Pre-Layout / Synthesis SDC Settings
# ============================================================

# Units
set_units -time ns -capacitance pF

# Clock definition
create_clock -name SYS_CLK -period 10 [get_ports SYS_CLK]

# Pre-CTS clock modeling
set_clock_latency  -source     2.0  [get_clocks SYS_CLK]
set_clock_latency              1.5  [get_clocks SYS_CLK]   ;# estimated tree delay
set_clock_uncertainty -setup   0.6  [get_clocks SYS_CLK]
set_clock_uncertainty -hold    0.3  [get_clocks SYS_CLK]
set_clock_transition           0.15 [get_clocks SYS_CLK]

# Wireload model
set_wire_load_model -name "5K_hvratio_1_4" -library slow
```

11.2 Post-Layout Constraints (STA Sign-off)

11.2.1 Characteristics

- Clocks are propagated (set_propagated_clock)
- Network latency removed — actual from SPEF
- Clock uncertainty reduced to jitter only (0.1 - 0.2ns)
- SPEF file back-annotated for actual wire parasitics
- OCV/AOCV/POCV derates applied

11.2.2 Post-Layout SDC Updates

```
# ============================================================
# Post-Layout / STA Sign-off SDC Changes
# ============================================================

# Switch to propagated clock mode
set_propagated_clock [all_clocks]
```

```
# Remove network latency (now from SPEF)
# DO NOT specify set_clock_latency (network) — remove that line

# Keep source latency (board delay)
set_clock_latency -source 2.0 [get_clocks SYS_CLK]

# Reduce uncertainty to jitter only
set_clock_uncertainty -setup 0.15 [get_clocks SYS_CLK]
set_clock_uncertainty -hold  0.05 [get_clocks SYS_CLK]

# Remove clock transition (now from SPEF)
# DO NOT specify set_clock_transition

# Apply OCV derates
set_timing_derate -late  1.05 [current_design]
set_timing_derate -early 0.95 [current_design]
```

11.3 SPEF Usage

11.3.1 What is SPEF?

SPEF (Standard Parasitic Exchange Format) is a file that contains the actual wire resistance and capacitance values extracted from the placed-and-routed layout. It is generated by parasitic extraction tools (Synopsys StarRC, Mentor Calibre xRC) and back-annotated into STA.

```
# In PrimeTime, read SPEF for back-annotation
read_parasitics -format SPEF \
    -complete_with zero \
    ../apr/output/design_top.postroute.spef

# For multiple corners (max = setup corner, min = hold corner)
read_parasitics -format SPEF -max ../spef/top_rcmax.spef
read_parasitics -format SPEF -min ../spef/top_rcmin.spef
```

11.4 Pre vs Post Comparison Table

SDC Element	Pre-Layout (Synthesis)	Post-Layout (Sign-off)
Clock mode	Ideal	Propagated
Clock tree delay	set_clock_latency	From SPEF (actual)
Clock uncertainty	0.5 - 1.0ns	0.1 - 0.2ns
Clock transition	set_clock_transition	From SPEF (actual)
Wire parasitics	Wireload models	SPEF back-annotation
OCV/AOCV/POCV	Optional	Mandatory for sign-off

SECTION SUMMARY

- ✓ Pre-layout constraints model the expected clock tree and use wireload models
- ✓ Post-layout constraints use propagated clocks and SPEF-annotated parasitics
- ✓ SPEF back-annotation provides accurate wire delays for post-route STA
- ✓ Clock uncertainty is reduced post-CTS as actual skew replaces estimates

Section 12: Constraint Validation and Debugging

After writing an SDC file, it is essential to validate the constraints and debug any issues before proceeding. Incorrect constraints are harder to debug than timing violations themselves.

12.1 check_timing

12.1.1 Theory

check_timing is a fundamental command that reports issues with timing constraints such as unconstrained paths, missing clocks, and incorrect exception usage. It should be run immediately after reading the SDC.

```
# Run check_timing to identify constraint issues
check_timing

# Common check_timing output messages to investigate:
# WARNING: No constrained paths
# WARNING: Input port <X> has no input delay constraint
# WARNING: Output port <Y> has no output delay constraint
# WARNING: Unconstrained endpoint in clock domain <Z>
# ERROR: Clock <A> not found at object <B>

# Check timing for specific path group
check_timing -verbose
```

12.1.2 Critical check_timing Warnings

Warning Message	What to Fix
No constrained paths found	Check if clock is properly defined on correct port
Unconstrained input ports	Add set_input_delay for all non-clock inputs
Unconstrained output ports	Add set_output_delay for all outputs
Clock source not found	Verify get_ports/get_pins target exists in netlist
Multiple clocks on same pin	Review create_generated_clock with -add flag

12.2 report_timing

12.2.1 Theory

report_timing is the primary command for examining individual timing paths. It shows the complete path from startpoint to endpoint with all cell delays, wire delays, and slack calculation.

```
# Basic timing report — worst setup path
report_timing -delay_type max

# Worst hold path
report_timing -delay_type min

# Report N worst paths
report_timing -delay_type max -max_paths 10

# Report path from specific register to specific register
report_timing -from [get_cells launch_ff] -to [get_cells capture_ff]

# Detailed report with all timing components
```

```
report_timing -path_type full_clock_expanded -delay_type max

# Report timing for a specific clock
report_timing -group SYS_CLK -delay_type max -max_paths 5
```

12.2.2 Understanding report_timing Output

```
# Sample report_timing output — reading guide

Startpoint: launch_ff/Q  (rising edge-triggered FF, clocked by SYS_CLK)
Endpoint:   capture_ff/D (rising edge-triggered FF, clocked by SYS_CLK)
Path Group: SYS_CLK
Path Type:  max (for setup check)

Point                                   Incr    Path
-----------------------------------------------------
clock SYS_CLK (rise edge)               0.00    0.00    <- Clock edge
clock network delay (ideal)             1.50    1.50    <- Clock tree delay
launch_ff/CK                            0.00    1.50 r  <- FF clock arrival
launch_ff/Q                             0.18    1.68 r  <- FF output delay
U1/A (NAND2X2)                          0.00    1.68 r  <- Net arrival at U1
U1/Y (NAND2X2)                          0.09    1.77 f  <- NAND delay (fall)
U2/A (INVX1)                            0.00    1.77 f
U2/Y (INVX1)                            0.06    1.83 r  <- INV delay (rise)
capture_ff/D                            0.00    1.83 r  <- Data arrival at D
data arrival time                                  1.83

clock SYS_CLK (rise edge)              10.00   10.00   <- Next capture edge
clock network delay (ideal)             1.50   11.50
capture_ff/CK                           0.00   11.50 r
library setup time                     -0.08   11.42   <- Setup time requirement
data required time                             11.42

data required time                             11.42
data arrival time                              -1.83
-------------------------------------------------
slack (MET)                                     9.59   <- Positive = timing met
```

12.3 report_constraints

```
# Report all design rule constraints and their status
report_constraint -all_violators

# Report max_transition violations
report_constraint -max_transition

# Report max_capacitance violations
report_constraint -max_capacitance

# Report max_fanout violations
report_constraint -max_fanout
```

12.4 Debugging Methodology

12.4.1 Debugging Unexpected Violations

5. Run check_timing — fix all unconstrained path warnings first
6. Run report_timing -max — identify worst paths
7. Examine if violation is in a path that should have an exception
8. Verify clock definitions with report_clocks
9. Check if path is through a false path with report_timing -path_type full

10. Verify clock domain assignments with report_clock_interaction

12.4.2 Additional Useful Commands

```
# List all clocks and their definitions
report_clocks

# Report all exceptions applied
report_exceptions

# Report clock interaction (which clocks share timing checks)
report_clock_interaction

# Report unconstrained paths
report_timing -unconstrained

# Show timing on a specific cell
report_timing -through [get_cells critical_cell]
```

SECTION SUMMARY

✓ check_timing must always be run first to validate constraint completeness
✓ report_timing provides detailed path analysis for debugging violations
✓ report_constraints identifies DRC violations (transition, cap, fanout)
✓ Use report_clocks and report_exceptions to verify constraint application

Section 13: Common Mistakes and Best Practices

This section consolidates the most frequent constraint mistakes seen in industry projects, along with best practices used by experienced timing engineers.

13.1 Over-Constraining

13.1.1 What is Over-Constraining?

Over-constraining means applying tighter timing requirements than the design actually needs. This forces the tool to use larger, faster cells unnecessarily, wasting area and power.

- Setting clock period too tight (e.g., 8ns when 10ns works)
- Adding excessive clock uncertainty
- Setting max_transition or max_capacitance too tight
- Applying multicycle path of 1 when 2 cycles is sufficient

 WARNING Over-constraining is extremely common in large teams where 'safety margin' is added at every level. At the block level, a 10% margin plus 10% at top-level means the design runs 20% faster than needed — massive area waste.

13.2 Under-Constraining

13.2.1 What is Under-Constraining?

Under-constraining means not applying enough constraints, allowing the tool to miss real violations. This is the most dangerous category — under-constrained designs pass STA but fail in silicon.

- Missing input or output delay constraints on some ports
- Not defining generated clocks on clock dividers or MUXes
- Missing clock uncertainty on cross-domain paths
- Omitting -min (hold) input/output delays
- Not specifying set_propagated_clock post-CTS

13.3 Wrong Timing Exceptions

13.3.1 Common Exception Mistakes

- Using set_false_path on real CDC paths instead of set_clock_groups
- Using set_false_path to 'fix' actual timing violations
- Multicycle path without adjusting hold — causes phantom hold violations
- Too-broad -through clause catching unintended paths
- False path on reset without verifying it is truly asynchronous

13.4 Industry Best Practices

13.4.1 Constraint Organization

11. Always define units at the top
12. Define all master clocks before generated clocks
13. Define clock_groups for all async clock pairs
14. Apply I/O constraints for all ports
15. Apply DRC constraints for the full design
16. Apply OCV derates before reporting timing

17. Run check_timing and resolve all warnings before optimization

13.4.2 Naming Conventions

```
# Use descriptive, consistent naming

# Clocks: <domain>_CLK suffix
create_clock -name SYS_CLK    [get_ports SYS_CLK]
create_clock -name USB_CLK    [get_ports USB_CLK]
create_clock -name PCIE_CLK   [get_ports PCIE_CLK]

# Generated clocks: GEN_ prefix or _DIVn suffix
create_generated_clock -name SYS_CLK_DIV2 ...

# Variables: lowercase with underscores
set clk_period  10.0
set input_delay  3.0
set output_delay 2.0

# Comments: explain WHY, not just WHAT
# Clock uncertainty: 0.6ns (pre-CTS): includes 0.3ns jitter + 0.3ns guard-band
```

13.4.3 Using Variables for Maintainability

```
# Define clock periods as variables — easier to update
set SYS_CLK_PERIOD  10.0   ;# 100 MHz
set FAST_CLK_PERIOD  4.0   ;# 250 MHz
set IO_SETUP         3.0
set IO_HOLD          0.5

create_clock -name SYS_CLK  -period $SYS_CLK_PERIOD  [get_ports SYS_CLK]
create_clock -name FAST_CLK -period $FAST_CLK_PERIOD [get_ports FAST_CLK]

set_input_delay -clock SYS_CLK -max $IO_SETUP [all_inputs]
set_input_delay -clock SYS_CLK -min $IO_HOLD  [all_inputs]
```

13.4.4 SDC Review Checklist

Item	Check
Units defined at top	set_units present
All clock ports have create_clock	run report_clocks
All generated clocks defined	check all dividers/MUXes
Clock uncertainty applied	both setup and hold
All I/O ports constrained	check_timing shows no unconstrained ports
Async clocks grouped	set_clock_groups for each async pair
MCP has hold adjustment	every -setup N has -hold N-1
OCV derates applied	set_timing_derate commands present
DRC constraints applied	max_transition, max_cap, max_fanout

SECTION SUMMARY

✓ Over-constraining wastes area and power — set realistic timing targets
✓ Under-constraining causes silicon failures — ensure all paths are covered

✓ Never use exceptions to hide real violations
✓ Use variables, consistent naming, and comments for maintainable SDC files
✓ Always run check_timing before optimization

Section 14: Complete Sample SDC File

The following is a complete, industry-standard SDC file for a representative ASIC design. This file demonstrates all major constraint categories with professional formatting and comprehensive comments.

14.1 Design Overview

The sample design has the following characteristics:

- Top module: PROCESSOR_TOP
- System clock: 100 MHz (10 ns period)
- Fast clock (PLL output): 250 MHz (4 ns period)
- USB clock: 60 MHz (asynchronous to system)
- Synchronous I/O bus, DDR output, async reset, scan chain

14.2 Complete SDC File

```
# ================================================================
# File:    PROCESSOR_TOP.sdc
# Design:  PROCESSOR_TOP
# Author:  Banothu Bapuji
# Qual:    M.Tech VLSI System Design
# Tool:    Synopsys PrimeTime / Design Compiler
# Rev:     1.0 (Pre-Layout)
# Date:    2025-01-01
#
# Description:
#   Full SDC constraint file for PROCESSOR_TOP.
#   Covers clocks, I/O timing, exceptions, CDC, DRC, and OCV.
# ================================================================

# ================================================================
# SECTION 1: UNITS
# ================================================================
set_units -time ns -capacitance pF -resistance kOhm -voltage V -current mA

# ================================================================
# SECTION 2: VARIABLES
# ================================================================
set SYS_CLK_PERIOD    10.0    ;# 100 MHz system clock
set FAST_CLK_PERIOD    4.0    ;# 250 MHz PLL output clock
set USB_CLK_PERIOD    16.67   ;# 60 MHz USB clock
set VIRT_CLK_PERIOD   10.0    ;# Virtual clock for I/O modeling

set CLK_UNCERTAINTY_SETUP  0.6   ;# Pre-CTS: jitter + guard-band
set CLK_UNCERTAINTY_HOLD   0.3
set CLK_TRANSITION         0.15
set CLK_SOURCE_LATENCY     2.0   ;# Board-level clock trace delay
set CLK_NETWORK_LATENCY    1.5   ;# Estimated clock tree delay (pre-CTS)

set IO_INPUT_DELAY_MAX    3.0
set IO_INPUT_DELAY_MIN    0.5
set IO_OUTPUT_DELAY_MAX   2.5
set IO_OUTPUT_DELAY_MIN   0.2
set IO_LOAD               5.0    ;# pF

# ================================================================
# SECTION 3: OPERATING CONDITIONS
```

```
# ================================================================
# Max (setup): Worst-case commercial corner
# Min (hold):  Best-case commercial corner
set_operating_conditions -max WCCOM -min BCCOM

# ================================================================
# SECTION 4: CLOCK DEFINITIONS
# ================================================================

# --- 4.1: Master System Clock (100 MHz) ---
create_clock -name SYS_CLK \
    -period $SYS_CLK_PERIOD \
    -waveform {0 5.0} \
    [get_ports SYS_CLK]

# --- 4.2: USB Clock (60 MHz, async to SYS_CLK) ---
create_clock -name USB_CLK \
    -period $USB_CLK_PERIOD \
    -waveform {0 8.335} \
    [get_ports USB_CLK]

# --- 4.3: Generated Clock from PLL output (250 MHz) ---
create_generated_clock -name FAST_CLK \
    -source [get_ports SYS_CLK] \
    -multiply_by 4 \
    [get_pins u_pll/CLKOUT]

# --- 4.4: Divided clock from flip-flop divider (50 MHz) ---
create_generated_clock -name CLK_DIV2 \
    -source [get_ports SYS_CLK] \
    -divide_by 2 \
    [get_pins u_clk_divider/Q]

# --- 4.5: Virtual clock for modeling I/O (no physical pin) ---
create_clock -name VIRT_CLK \
    -period $VIRT_CLK_PERIOD

# ================================================================
# SECTION 5: CLOCK ATTRIBUTES (PRE-CTS)
# ================================================================

# Source latency (board trace delay) — keep in post-layout SDC too
set_clock_latency -source $CLK_SOURCE_LATENCY [get_clocks {SYS_CLK USB_CLK}]

# Network latency (estimated clock tree — REMOVE after CTS)
set_clock_latency $CLK_NETWORK_LATENCY [get_clocks {SYS_CLK USB_CLK FAST_CLK}]

# Clock uncertainty
set_clock_uncertainty -setup $CLK_UNCERTAINTY_SETUP [get_clocks {SYS_CLK FAST_CLK CLK_DIV2}]
set_clock_uncertainty -hold  $CLK_UNCERTAINTY_HOLD  [get_clocks {SYS_CLK FAST_CLK CLK_DIV2}]

# Tighter uncertainty on USB clock (reference IC)
set_clock_uncertainty -setup 0.4 [get_clocks USB_CLK]
set_clock_uncertainty -hold  0.2 [get_clocks USB_CLK]

# Clock transition (pre-CTS only — remove after CTS)
set_clock_transition $CLK_TRANSITION [all_clocks]

# ================================================================
# SECTION 6: CLOCK DOMAIN GROUPS
# ================================================================

# SYS_CLK domain and USB_CLK are completely asynchronous
```

```
set_clock_groups -asynchronous \
    -group {SYS_CLK FAST_CLK CLK_DIV2} \
    -group {USB_CLK}

# ================================================================
# SECTION 7: INPUT CONSTRAINTS
# ================================================================

# Collect all input ports except clock and reset
set non_clk_inputs [remove_from_collection \
    [all_inputs] \
    [get_ports {SYS_CLK USB_CLK RESET_N SCAN_EN}]]

# Input delay (data valid 3ns after clock edge)
set_input_delay -clock SYS_CLK -max $IO_INPUT_DELAY_MAX $non_clk_inputs
set_input_delay -clock SYS_CLK -min $IO_INPUT_DELAY_MIN $non_clk_inputs

# Input transition (drive quality from external device)
set_input_transition 0.2 $non_clk_inputs

# Stricter driving cell for critical control inputs
set_driving_cell -lib_cell BUF4X [get_ports {cfg_addr[*] cfg_data[*]}]

# USB port inputs — constrained relative to USB clock
set_input_delay -clock USB_CLK -max 2.0 [get_ports {usb_data[*] usb_valid}]
set_input_delay -clock USB_CLK -min 0.3 [get_ports {usb_data[*] usb_valid}]

# ================================================================
# SECTION 8: OUTPUT CONSTRAINTS
# ================================================================

# Output delay (external device needs data 2.5ns before clock edge)
set_output_delay -clock SYS_CLK -max $IO_OUTPUT_DELAY_MAX [all_outputs]
set_output_delay -clock SYS_CLK -min $IO_OUTPUT_DELAY_MIN [all_outputs]

# Output load (5pF external capacitance on all outputs)
set_load $IO_LOAD [all_outputs]

# ================================================================
# SECTION 9: TIMING EXCEPTIONS
# ================================================================

# --- 9.1: False Paths ---

# Asynchronous reset — only active during power-up/reset sequence
set_false_path -from [get_ports RESET_N]

# Scan enable — only active in test mode, not in functional timing
set_false_path -from [get_ports SCAN_EN]

# --- 9.2: Multicycle Paths ---

# Configuration register — updated once every 8 SYS_CLK cycles
# Data is guaranteed stable for 8 cycles after write
set_multicycle_path -setup 8 \
    -from [get_cells {u_cfg_reg/*}] \
    -to   [get_cells {u_datapath/*}]
set_multicycle_path -hold  7 \
    -from [get_cells {u_cfg_reg/*}] \
    -to   [get_cells {u_datapath/*}]

# Slow status register — sampled every 4 cycles
```

```
set_multicycle_path -setup 4 -from [get_cells u_status_reg] -to [get_cells u_cpu_core]
set_multicycle_path -hold  3 -from [get_cells u_status_reg] -to [get_cells u_cpu_core]

# --- 9.3: CDC Max Delay ---

# 2-FF synchronizer input: constrain combinational path only
# (clock_groups already removes clocked check; this budgets combo logic)
set_max_delay -datapath_only 4.0 \
    -from [get_clocks SYS_CLK] \
    -to   [get_cells {u_cdc_sync_usb/ff1}]

set_max_delay -datapath_only 8.0 \
    -from [get_clocks USB_CLK] \
    -to   [get_cells {u_cdc_sync_sys/ff1}]

# ================================================================
# SECTION 10: DESIGN RULE CONSTRAINTS
# ================================================================

# Maximum transition: 0.5ns on entire design, 0.2ns on clock
set_max_transition 0.5 [current_design]
set_max_transition 0.2 [get_clocks {SYS_CLK FAST_CLK}]

# Maximum capacitance: 0.5pF
set_max_capacitance 0.5 [current_design]

# Maximum fanout: 32 (tool will buffer high-fanout nets)
set_max_fanout 32 [current_design]

# ================================================================
# SECTION 11: OCV DERATES (PRE-LAYOUT — CONSERVATIVE)
# ================================================================

# 5% late derate (setup), 5% early derate (hold)
set_timing_derate -late  1.05 [current_design]
set_timing_derate -early 0.95 [current_design]

# ================================================================
# SECTION 12: CASE ANALYSIS
# ================================================================

# In functional mode analysis, scan_en = 0, test_mode = 0
set_case_analysis 0 [get_ports SCAN_EN]
set_case_analysis 0 [get_ports TEST_MODE]

# ================================================================
# END OF SDC FILE
# ================================================================
```

SECTION SUMMARY

- ✓ Complete SDC includes: units, variables, clocks, uncertainty, I/O, exceptions, DRC, OCV
- ✓ Variables make the SDC maintainable — change one number, all references update
- ✓ Comments explain WHY each constraint exists, not just what it does
- ✓ Every exception should have a comment explaining the design intent

Section 15: Interview Questions & Answers

This section provides a comprehensive set of SDC and STA interview questions from beginner to advanced level, including scenario-based questions frequently asked in VLSI industry interviews.

15.1 Basic Level Questions

Q1. What does SDC stand for and what is its purpose?

Answer: SDC stands for Synopsys Design Constraints. It is a Tcl-based constraint file format used to specify timing, power, and area requirements for digital designs in synthesis and static timing analysis. SDC tells EDA tools the clock frequencies, I/O timing requirements, timing exceptions, and physical constraints needed to optimize and verify the design.

Q2. What is the difference between setup time and hold time?

Answer: Setup time (Tsetup) is the minimum time the data must be stable BEFORE the active clock edge. Hold time (Thold) is the minimum time the data must remain stable AFTER the active clock edge. Violating setup time causes data to not be captured correctly; violating hold time causes incorrect (metastable or wrong) data to be captured.

```
Setup check: Data arrival time + Tsetup <= Clock arrival time (next edge)
Hold check:  Data arrival time >= Clock arrival time (same edge) + Thold
```

Q3. What is clock uncertainty and what does it model?

Answer: Clock uncertainty is a guard-band applied to clock arrival times to model jitter (cycle-to-cycle variation), clock skew (different arrival times at different FFs), and additional safety margin. Setup uncertainty tightens the setup check; hold uncertainty tightens the hold check. Pre-CTS uncertainty is larger (0.5-1.0ns) than post-CTS uncertainty (0.1-0.2ns).

Q4. What is a virtual clock? When is it used?

Answer: A virtual clock is defined with create_clock but has no physical clock source (no [get_ports] argument). It is used to model an external clock that does not enter the chip but drives I/O timing. For example, if an external processor and this chip share a 100MHz clock but the clock only goes to the external device — define a virtual 100MHz clock and reference I/O constraints to it.

Q5. What is the difference between set_false_path and set_multicycle_path?

Answer: set_false_path removes the timing check entirely — the path is treated as if it doesn't need to meet any timing. Use this for paths that have no real functional timing requirement (async reset, test paths). set_multicycle_path relaxes the timing check by N clock cycles — the path still exists and must meet timing, but over N cycles instead of 1. Use this when data legitimately takes multiple clock cycles.

15.2 Intermediate Level Questions

Q6. When you set set_multicycle_path -setup 2, what must you also set and why?

Answer: You must also set set_multicycle_path -hold 1. By default, the hold check examines the 0-cycle edge (same edge). When MCP setup = 2, the capture edge moves forward 2 cycles. Without adjusting hold, the hold check would be checked at the N-1 cycle back edge — which is extremely pessimistic and would cause false hold violations. Setting hold to N-1 moves the hold check to the correct reference edge.

Q7. What is the difference between set_clock_groups -asynchronous and set_false_path between clocks?

Answer: Functionally they both prevent timing analysis between the two clock domains. However, set_clock_groups is the preferred method because: (1) it is semantically clearer and communicates design intent, (2) it is bidirectional by default, (3) it is recognized by CDC verification tools, (4) it is easier to maintain when multiple clock pairs need to be grouped. set_false_path -from CLK_A -to CLK_B is unidirectional and requires a second command for the reverse path.

Q8. Explain the concept of ideal clock vs propagated clock.

Answer: An ideal clock has zero latency and zero transition through the clock tree — all FFs see the clock arrive at exactly the same time as the source. Pre-CTS, clocks are ideal (with manually added latency/uncertainty). A propagated clock uses the actual physical clock tree delays computed from the netlist parasitics (SPEF). Post-CTS, set_propagated_clock is used to switch to propagated mode, enabling the tool to compute real clock skew and arrival times at each FF.

Q9. What is -datapath_only in set_max_delay?

Answer: The -datapath_only flag with set_max_delay completely replaces the clock period with the specified delay value for setup analysis. No clock uncertainty is applied, and the check is purely on the combinational data path delay. This is commonly used for CDC paths through synchronizers where you want to budget the logic delay before the synchronizer FF without any clock relationship. Without -datapath_only, set_max_delay still adjusts relative to the clock period.

Q10. What is OCV and why is it important in advanced nodes?

Answer: OCV (On-Chip Variation) accounts for the fact that identical cells on the same die, at different physical locations, experience slightly different delays due to process variation, voltage droop, and temperature gradients. In advanced nodes (28nm and below), this within-die variation is significant. OCV applies a derate factor — late paths are slowed by (1 + derate%) for setup analysis, and early paths are sped up by (1 - derate%) for hold analysis. This creates a realistic worst-case scenario.

15.3 Advanced Level Questions

Q11. How would you handle a design with a 200MHz clock driving logic that feeds into a 100MHz domain?

```
# Method 1: set_clock_groups if domains are async
set_clock_groups -asynchronous -group {CLK_200} -group {CLK_100}

# Method 2: If synchronous (2:1 ratio, use MCP)
set_multicycle_path -setup 2 -from [get_clocks CLK_200] -to [get_clocks CLK_100]
set_multicycle_path -hold  1 -from [get_clocks CLK_200] -to [get_clocks CLK_100]
```

```
# Method 3: If using a 2-FF synchronizer (semi-synchronous CDC)
set_max_delay -datapath_only [expr {1/$clk100_freq * 0.8}] \
    -from [get_clocks CLK_200] -to [get_cells {*sync_ff*}]
```

Q12. What is a clock gating check? What happens if it fails?

Answer: A clock gating check verifies that the enable signal of a clock gate (ICG cell) meets setup/hold-like requirements relative to the clock it gates. The enable must be stable before the clock edge that would pass through the gate. If the gating check fails (enable transitions too close to the clock edge), it can cause a glitch on the gated clock — a partial pulse that looks like a clock edge to downstream flip-flops, causing incorrect data capture. This is a critical functional bug in power-gated designs.

Q13. How would you debug a hold violation that appears only in post-route STA but not in synthesis?

Answer: This scenario typically occurs when: (1) The clock tree was not balanced properly (high skew) and the capture clock arrives much earlier than the launch clock. (2) Post-route wire parasitics are much lower than wireload models (fast data path). (3) OCV derate factors applied post-route — early derate speeds up the capture path. Debug steps: examine the clock skew with report_timing -path_type full_clock_expanded, check if the hold violation is on a short data path, verify that set_propagated_clock is active, check OCV settings, and inspect if the hold exception MCP adjustment was missed.

Q14. What is the significance of -add in create_generated_clock?

Answer: The -add flag allows multiple generated clocks to be defined on the same physical pin. This is needed when a clock MUX can select between two different clocks — both clocks exist on the MUX output pin simultaneously in the SDC model. Without -add, defining a second clock on the same pin would overwrite the first. With -add, both clocks coexist on the pin, and the tool analyzes timing under both scenarios. This is critical for MUX-selected clock scenarios to prevent unconstrained paths under one operating mode.

Q15. Scenario: Your design has a configuration register that is written every 100 clock cycles and read by a high-speed datapath every cycle. How do you constrain this?

Answer: Use set_multicycle_path for the path from the configuration register to the datapath. Since the config register is guaranteed stable for 100 cycles, a multicycle path of 100 is appropriate. However, in practice, a smaller multicycle (e.g., 2-4) is used to keep some timing margin and avoid overly long paths. Example:

```
# Config register written every 100 cycles — data stable for 100 cycles
# Use MCP = 2 for practical purposes (don't need full 100 relaxation)
set_multicycle_path -setup 2 \
    -from [get_cells u_cfg_reg/*] \
    -to   [get_cells u_datapath/*]
set_multicycle_path -hold  1 \
    -from [get_cells u_cfg_reg/*] \
    -to   [get_cells u_datapath/*]

# If the config register is in a different clock domain:
set_clock_groups -asynchronous \
    -group {CFG_CLK} \
    -group {DATA_CLK}
```

Q16. What is POCV and how does it differ from OCV?

Answer: OCV applies a fixed flat derate to all paths — a pessimistic but simple model. POCV (Parametric OCV) uses statistical characterization data from the library — each cell has a mean delay and a sigma (standard deviation). The STA tool computes the statistical sum of all cell delays along a path and applies a confidence-level-based guard band (typically 3-sigma). POCV is less pessimistic than OCV because statistical averaging along longer paths naturally reduces the combined sigma. POCV is the required methodology for 16nm and below process nodes.

Q17. How do you verify that all CDC crossings are properly constrained?

```
# Step 1: Run check_timing to find unconstrained paths
check_timing -verbose

# Step 2: Run report_clock_interaction to see all clock pairs
report_clock_interaction

# Step 3: Check if any 'tns' (total negative slack) exists in cross-domain
report_timing -from [get_clocks CLK_A] -to [get_clocks CLK_B]

# Step 4: Verify clock groups are comprehensive
report_clock_groups

# Step 5: Use SpyGlass CDC or Questa CDC for complete CDC verification
```

15.4 Quick Reference — Key SDC Commands

Command	Primary Use
create_clock	Define master clock on port/pin
create_generated_clock	Define derived clock from divider/MUX/PLL
set_clock_latency	Model source (board) and network (tree) delay
set_clock_uncertainty	Apply jitter and skew guard-band
set_clock_transition	Model clock slew (pre-CTS only)
set_propagated_clock	Switch to actual clock tree delays (post-CTS)
set_input_delay	Model external data arrival timing at inputs
set_output_delay	Model external capture timing at outputs
set_driving_cell	Model external driver for input ports
set_load	Model external capacitive load at output ports
set_false_path	Remove timing check on irrelevant paths
set_multicycle_path	Relax timing for multi-cycle data paths
set_max_delay	Override setup check with specific budget
set_min_delay	Override hold check with specific budget
set_clock_groups	Define async/exclusive clock domain groups

set_max_transition	Limit signal slew rate (DRC)
set_max_capacitance	Limit capacitive load per driver (DRC)
set_max_fanout	Limit gate output fanout (DRC)
set_timing_derate	Apply OCV/AOCV variation derates
set_case_analysis	Fix port/pin values for a specific operating mode
set_units	Define time, capacitance, and other units
check_timing	Validate constraint completeness
report_timing	Report path-level timing details

SECTION SUMMARY

✓ Master the 5 categories: Clocks, I/O, Exceptions, CDC, DRC
✓ Understand WHY each constraint exists, not just its syntax
✓ Always pair setup MCP with hold MCP (N-1)
✓ Use set_clock_groups for async domains; not set_false_path
✓ Debug flow: check_timing → report_clocks → report_timing → report_exceptions

'SDC Constraints – Complete Practical & Theoretical Guide'

About the Book

This book provides a complete and practical understanding of Synopsys Design Constraints (SDC) used in VLSI design. Covering both theoretical concepts and real-world implementation, it enables engineers to confidently write and debug timing constraints for ASIC and FPGA designs. From basic clock definitions to advanced timing exceptions and variation modeling, this guide serves as a one-stop resource for mastering Static Timing Analysis (STA) constraints.

Key Features

- **Complete coverage** from basics to advanced SDC concepts
- **Real industry examples** and syntax explanations
- **Detailed clock, IO,** and **exception constraints**
- **Practical debugging and validation techniques**
- **Coverage of OCV, AOCV,** and **POCV**
- **Interview questions** and **real-world scenarios**

About the Author

Banothu Bapuji
M.Tech VLSI System Design

'**Banothu Bapuji** is a VLSI enthusiast specializing in timing analysis, ASIC design flows, and constraint development. With a strong focus on practical learning, this book is designed to simplify complex SDC concepts for engineers and students.'